IPS 6 **Hennetier Antoine**

Méthodologie de Pentest

.

Contacts :

Maître d'apprentissage : Hugo Bourcier

Crédit Agricole côte d'azur

Date de publication :

10/12/2025

Table des matières

Prérequis : 3

Contrat : 4

Collectes d'informations : 9

Recherche de menaces : 15

Analyse des vulnérabilités : 22

Exploitation : 29

Post-Exploitation : 36

Rapport : 42

Prérequis :

La méthodologie de test d'intrusion est issue de la PTES (Pénétration Testing Execution Standard). Elle se décompose en 7 phases :

1. Contrat
2. Collecte de renseignements
3. Recherche des menaces
4. Analyse des vulnérabilités
5. Exploitation
6. Post-Exploitation
7. Rapport

C'est une méthodologie américaine officielle, disponible sous ce lien :

http://www.pentest-standard.org/index.php/Main_Page

Contrat :

1. Introduction : l'importance d'un cadrage contractuel solide

Dans tout test d'intrusion, la réussite de l'opération repose autant sur la qualité technique du test que sur la solidité de la phase de préparation. Pourtant, cette phase pré-engagement est souvent négligée. Définir correctement la portée, les objectifs et les responsabilités contractuelles permet non seulement d'éviter les incompréhensions et la dérive de périmètre, mais aussi de protéger juridiquement l'entreprise et les testeurs.

Le contrat et les documents associés (SOW, NDA, autorisation de test) constituent le cadre légal dans lequel le test sera réalisé. Ils servent de référence en cas de litige, garantissent la conformité et définissent précisément ce qui est autorisé et ce qui ne l'est pas.

2. Définition de la portée : un pilier contractuel

La portée (scope) détermine **ce qui doit être testé, comment, et dans quelles limites**. Elle est définie conjointement avec le client, généralement lors de la phase de négociation et de cadrage.

2.1 Contenu typique de la portée

- Ressources à tester : IP, applications, environnements, segments réseau.
- Ressources explicitement exclues.
- Type de test : boîte noire, grise, blanche.
- Méthodes autorisées ou interdites.
- Objectifs prioritaires (ex. : compromission, analyse défensive, validation d'un patch, etc.).

Une portée claire évite les ambiguïtés telles que :

- Le client qui s'attend à une attaque exhaustive alors qu'il n'a fourni qu'un budget minimal ;
- Le testeur qui dépasse involontairement les limites légales en explorant des environnements non autorisés.

3. Structure de coûts et impact contractuel

3.1 L'importance de relier le coût au volume de travail réel

Le contrat doit clairement expliquer comment le coût est construit.
Exemple classique :

- 100 IP pour 100 000 € = 1 000 € par IP
  Mais ce modèle devient incohérent lorsqu'il s'agit d'une application critique unique, nécessitant beaucoup plus de temps d'analyse.

Le coût doit être aligné sur l'effort, et non sur un ratio mécanique.
Sans cela :

- Risque de sous-facturation,
- Perte de rentabilité,
- Réduction involontaire de la qualité du test.

3.2 Flexibilité et accompagnement du client

Le contrat peut prévoir une clause indiquant que le testeur aidera le client à clarifier ses besoins, notamment si celui-ci :

- Ne connaît pas ses priorités,
- Ne sait pas ce qui doit être testé,
- Ne maîtrise pas la dimension technique de la sécurité.

Le rôle du testeur inclut donc un devoir de conseil.

4. Estimation du temps et rembourrage contractuel

4.1 Méthodologie d’estimation

L’estimation du temps dépend :

- De l’expérience du testeur,
- Des engagements similaires passés,
- De la complexité du périmètre.

Une fois l’estimation effectuée, il est recommandé d’ajouter environ **20 % de marge** (rembourrage ou "consulting overhead").

4.2 Justification du rembourrage

La marge permet d’absorber :

- Des interruptions (pannes réseau, maintenance imprévue) ;
- La découverte d’une vulnérabilité critique nécessitant de longues réunions ;
- Des contraintes organisationnelles côté client.

Si la marge n’est pas utilisée, le testeur doit fournir **de la valeur supplémentaire**, par exemple :

- Approfondissement des analyses,
- Mini-présentation technique,
- Rapport complémentaire.

Facturer du temps non travaillé serait contraire à l’éthique et pourrait causer des litiges.

4.3 Dates de début et de fin

Tout contrat doit inclure :

- Une **date de début**,

- Une **date limite de fin**,
- Les conditions d'extension du test et le coût horaire applicable.

Cela protège le testeur contre la dérive du périmètre et clarifie les attentes du client.

5. Réunion de cadrage (Kick-off)

La réunion de cadrage, généralement post-signature du contrat (ou sous NDA), sert à clarifier précisément la portée.

5.1 Objectifs

- Valider les hypothèses établies dans la proposition.
- Confirmer les IP, applications, environnements concernés.
- Identifier les tiers impliqués (DNS, firewall, cloud provider…).
- Relever les contraintes légales selon les pays (RGPD, données personnelles, etc.).

Les **règles d'engagement** (ROE) et **les coûts** ne doivent pas être discutés ici pour éviter la confusion — chaque sujet mérite une réunion dédiée.

6. Travail hors périmètre et gestion contractuelle

6.1 Risques du travail ad hoc

Effectuer du travail non prévu sans documents contractuels :

- Crée une dérive de périmètre,
- Génère des coûts non anticipés,
- Augmente le risque juridique (absence de preuve d'autorisation).

6.2 Processus recommandé

Toute demande additionnelle doit :

1. Être formalisée dans un **SOW (Statement of Work)** dédié,
2. Décrire précisément le travail supplémentaire,
3. Être signé par les deux parties,
4. Mentionner le tarif horaire ou forfaitaire applicable.

Le contrat principal doit prévoir une clause spécifique permettant ce type de travaux additionnels.

7. Questionnaires pré-engagement

Afin d'établir la portée contractuelle, le testeur doit poser une série de questions au client, par exemple :

7.1 Questions générales

- Quel est l'objectif principal du test ?
- Pourquoi souhaitez-vous effectuer un test d'intrusion maintenant ?
- Quels sont vos cas d'usage prioritaires ?

7.2 Questions techniques

- Quels environnements doivent être testés ?
- Quels types d'attaques sont autorisés ou interdits ?
- Y a-t-il des contraintes temporelles ou opérationnelles ?

7.3 Questions juridiques et organisationnelles

- Des tiers gèrent-ils certaines infrastructures ?
- Avez-vous des contraintes légales spécifiques (ex. RGPD) ?
- Quels pays/juridictions sont concernés ?

Collectes d'informations :

1. Introduction à la collecte d'informations

La collecte d'informations a pour objectif d'obtenir une compréhension globale du système, du réseau ou de l'organisation ciblée. Elle vise à identifier :

- La surface d'attaque exposée,
- Les technologies utilisées,
- Les dépendances,
- Les personnes ou entités clés,
- Les failles potentielles dans les configurations, les protocoles ou les pratiques internes.

Cette phase se déroule **avant toute activité intrusive**, et peut être réalisée **avec ou sans interaction directe avec la cible**, en fonction du type de test (boîte noire, grise ou blanche) et des règles d'engagement.

2. Les deux grandes catégories de collecte d'informations

2.1 Collecte passive (Passive Reconnaissance)

La collecte passive consiste à rassembler des informations **sans interagir directement avec les systèmes de la cible**. Elle n'est donc pas détectable et imite le comportement d'un attaquant réel qui cherche à éviter toute alerte.

Objectifs principaux

- Identifier les actifs visibles publiquement,
- Comprendre la structure de l'entreprise,
- Relever les technologies et versions exposées indirectement,
- Obtenir des données sur les employés ou partenaires.

Sources typiques

- **Moteurs de recherche** : Google, Bing, Yandex, Shodan, Censys
- **Archives Internet** : Wayback Machine
- **Registres publics** : WHOIS, RIPE, ARIN
- **Réseaux sociaux** : LinkedIn, Facebook, Twitter
- **Fuites de données** (dumps publics, paste sites)
- **Communiqués de presse, rapports financiers, offres d'emploi**
- **Analyse DNS passive**
- **Données OSINT (Open Source Intelligence)**

Compléments possibles

- Recherche d'empreintes numériques (fingerprinting indirect),
- Identification d'adresses e-mail et d'utilisateurs,
- Collecte de documents exposés contenant des métadonnées (PDF, DOCX, etc.),
- Cartographie de la présence cloud (AWS, Azure, GCP, SaaS...).

La collecte passive permet de minimiser les risques légaux et opérationnels tout en fournissant des informations de grande valeur.

2.2 Collecte active (Active Reconnaissance)

La collecte active implique **une interaction directe avec la cible**, ce qui peut être détecté par les IDS/IPS, les firewalls ou les équipes de sécurité. Elle doit être strictement conforme aux règles d'engagement et aux autorisations légales obtenues.

Objectifs principaux

- Identifier les services ouverts,
- Déterminer les versions et configurations,
- Découvrir les points d'entrée potentiels pour une exploitation.

Techniques courantes

- **Scanning de ports** : Nmap, Masscan
- **Analyse de services** : bannering, detecion de versions
- **Enumération DNS active** : DNS zone transfer, brute force de sous-domaines
- **Enumération SMB, LDAP, SNMP, RPC**
- **Détection d'hôtes** (ICMP, ARP, traceroute)
- **Analyse de certificats SSL/TLS**
- **Enumération d'applications Web** : dirbusting, crawling, fingerprinting

Risques associés

- Alerte des systèmes de défense,
- Blocage d'IP,
- Interruption possible de service si les actions sont mal configurées,
- Bruit réseau significatif.

La collecte active complète la collecte passive en apportant une connaissance réelle et vérifiable des systèmes.

3. Objectifs opérationnels de la collecte d'informations

La collecte d'informations doit permettre de :

3.1 Construire une cartographie claire de la surface d'attaque

Incluant :

- Adresses IP,
- Domaines et sous-domaines,
- Services exposés,
- Technologies en place,
- Environnements cloud.

3.2 Identifier les vulnérabilités potentielles

Même si l'identification approfondie des vulnérabilités est une étape ultérieure, la reconnaissance permet déjà de repérer :

- Protocoles obsolètes,
- Services non sécurisés,
- Publications involontaires de données,
- Mauvaises pratiques (ex : WordPress non à jour, services admin exposés).

3.3 Préparer les scénarios d'attaque

Les informations obtenues orientent :

- Les vecteurs d'attaque les plus probables,
- Les priorités d'exploitation,
- La faisabilité des attaques (locales, distantes, internes, humaines...).

4. Méthodologie structurée : du général au spécifique

Une approche progressive garantit une collecte efficace, sans bruit excessif ni perte de temps.

Étape 1 : Analyse documentaire

Recherche d'informations publiques et contextuelles.

Étape 2 : Cartographie des actifs

Découverte des IP, DNS, applications, services.

Étape 3 : Fingerprinting technique

Identification des systèmes : OS, services, versions, Framework.

Étape 4 : Enumération ciblée

SMB, SNMP, FTP, SSH, Web apps, API…

Étape 5 : Consolidation et analyse

Création du graphe d'attaque et planification des tests.

5. Importance des contraintes légales et contractuelles

La collecte d'informations, surtout active, doit être encadrée par :

- Une **autorisation de test formelle**,
- Des **Règles d'Engagement (ROE)** définissant précisément :
    - Ce qui est autorisé,
    - Ce qui est interdit,
    - Les plages horaires,
    - Les systèmes sensibles à exclure,
    - Les contacts d'urgence.

Même certaines activités passives peuvent poser un problème dans des juridictions strictes (ex. collecte sur des espaces cloud partagés).

6. Livrables liés à la collecte d'informations

Dans un rapport de test d'intrusion, la reconnaissance doit être documentée dans une section dédiée, incluant :

- Liste des actifs identifiés,
- Méthodes utilisées,
- Détails de la surface d'attaque,
- Cartes réseau ou schémas,
- Preuves de découvertes pertinentes,
- Risques associés à la surface exposée.

Cela permet au client de comprendre :

- L'étendue réelle de son exposition,
- Les faiblesses structurelles,
- Les priorités pour réduire la surface d'attaque.

7. Erreurs courantes à éviter

- Confondre collecte passive et active (risque de détection).
- Générer trop de bruit réseau.
- Tester des systèmes non autorisés.
- Oublier les environnements cloud ou SaaS.
- Se limiter à un seul outil (Nmap ≠ reconnaissance complète).

- Sous-documenter la phase, rendant le rapport final incomplet.

Recherche de menaces :

1. Introduction à la recherche des menaces

La recherche des menaces (Threat Intelligence) consiste à collecter, analyser et interpréter des informations concernant :

- Les attaquants potentiels,
- Leurs motivations,
- Les techniques qu'ils emploient,
- Les vulnérabilités connues ou émergentes,
- Les failles exploitables dans le contexte de la cible.

Cette analyse permet d’adapter la méthodologie à la réalité du risque et d’orienter les tests vers les scénarios les plus crédibles, plutôt que de se limiter à une approche technique générique.

2. Objectifs de la recherche des menaces dans un test d'intrusion

La recherche des menaces sert à :

2.1 Identifier les risques pertinents

- Menaces spécifiques au secteur (ex. : attaques sur le secteur bancaire, hospitalier, industriel).
- Groupes d'attaquants connus pour cibler ce type d'organisation.
- Vulnérabilités exposées publiquement (ex. : CVE récentes) susceptibles d'affecter l'environnement.

2.2 Contextualiser la surface d'attaque

À partir de la phase de reconnaissance, il s'agit de déterminer :

- Les technologies présentes et les failles qui les affectent,
- La présence potentielle de données sensibles,
- Les dépendances tierces pouvant introduire un risque.

2.3 Prioriser les scénarios d'attaque

Les attaques les plus probables ou les plus dangereuses deviennent prioritaires :

- Exploitation d'une CVE critique récente,
- Attaques par ingénierie sociale réalistes,
- Compromission de comptes exposés sur le Web,
- Attaques ciblant des protocoles internes, etc.

2.4 Simuler des menaces réelles

Plutôt que de tester "tout", le testeur peut simuler :

- Un attaquant opportuniste,
- Un attaquant interne malveillant,
- Un groupe APT,
- Un individu ciblant un type de données particulier.

3. Sources et types d’informations utilisées

La recherche des menaces exploite plusieurs types d’informations.

3.1 Threat Intelligence ouverte (OSINT)

- Bases CVE et NVD
- Bulletins de sécurité (cert.gov, ANSSI, CISA, CIS, vendor advisories…)
- Portails spécialisés (MITRE ATT&CK, Exploit-DB, VulDB…)
- Recherches de vulnérabilités spécifiques aux technologies repérées
- Forums et réseaux sociaux des communautés de cybersécurité

3.2 Threat Intelligence commerciale

Si le client ou le testeur possède un abonnement :

- Flux d’IOC, signatures, IP malveillantes,
- Analyses d’APT et de campagnes en cours,
- Rapports sectoriels spécifiques.

3.3 Intelligence interne au client

- Incidents passés,
- Logs existants,
- Rapports d’audit,
- Incidents remontés par les équipes SOC/Blue Team,
- Échantillons de phishing reçus.

3.4 Analyses techniques issues de la reconnaissance

- Technologies détectées,
- Services exposés,

- Framework ou versions identifiées,
- Empreintes numériques.

4. Méthodologie structurée de recherche des menaces

La méthodologie peut être divisée en plusieurs étapes clés.

Étape 1 : Collecte initiale d'informations

À partir :

- Du client,
- De la reconnaissance passive,
- Des sources publiques,
- De la veille sécurité.

Objectif : déterminer ce qui pourrait affecter la cible.

Étape 2 : Classification des menaces

Les menaces sont classées selon :

- Probabilité,
- Impact,
- Exploitabilité,
- Facilité d'accès à la cible.

Méthodes utilisées :

- Modèles STRIDE, DREAD,
- Cartographie OWASP,
- Matrice MITRE ATT&CK.

Étape 3 : Analyse de la pertinence

Chaque menace doit être confrontée à :

- L'environnement réel,
- Les objectifs du test,
- Les ressources du testeur (temps, budget).

Cette étape permet d'isoler les menaces réalistes à tester.

Étape 4 : Définition des scénarios d'attaque

Les scénarios sont construits pour simuler les menaces identifiées :

- Accès initial (phishing, scan, exploitation CVE),
- Élévation de privilège,
- Mouvement latéral,
- Exfiltration de données.

Étape 5 : Intégration au plan de test

Les scénarios réalistes deviennent :

- Des cas de test,
- Des phases spécifiques de l'audit,
- Ou des objectifs prioritaires.

5. Exemples pratiques de menaces identifiées

5.1 Exemple 1 : Technologie non patchée

Une version d'Apache vulnérable repérée →
→ menace : exploitation d'une CVE critique
→ scénario : gain d'accès à la machine

5.2 Exemple 2 : Dopage social

Employés exposant des données sur LinkedIn →
→ menace : phishing ciblé
→ scénario : compromission d'un compte bureautique

5.3 Exemple 3 : Exposition Cloud

Sous-domaine pointant vers AWS S3 public →
→ menace : fuite de données
→ scénario : récupération ou modification de fichiers

6. Intégration avec les phases du test d'intrusion

La recherche des menaces est transversale :
elle commence avant le test et continue tout au long de l'audit.

Avant le test

- Compréhension du contexte
- Définition des risques critiques
- Sélection des scénarios

Pendant le test

- Ajustement de la stratégie selon les découvertes
- Analyse dynamique des nouvelles surfaces d'exposition
- Recherche de vulnérabilités en fonction de technologies découvertes

Après le test

- Réévaluation des menaces résiduelles,
- Documentation dans le rapport,
- Recommandations adaptées au niveau de risque.

7. Livrables liés à la recherche des menaces

Dans un rapport professionnel, la section sur les menaces doit inclure :

- Liste des menaces pertinentes pour la cible,
- Matrice de risque (probabilité × impact),
- Scénarios construits à partir des menaces,
- Liens avec les vulnérabilités découvertes,
- Recommandations de réduction ou mitigation.

8. Principales erreurs à éviter

- Se limiter à une liste générique de menaces.
- Ignorer le contexte sectoriel du client.
- Tester des scénarios irréalistes ou hors périmètre.
- Ne pas utiliser la reconnaissance comme base.
- Se focaliser uniquement sur les CVE, sans tenir compte des menaces humaines (ingénierie sociale).
- Sous-estimer les menaces internes.

Analyse des vulnérabilités :

1. Objectifs de l’analyse des vulnérabilités

L’analyse vise à :

- Identifier les failles techniques (CVE, mauvaises configurations, services obsolètes).
- Mesurer leur gravité (score CVSS, impact, exploitabilité).
- Déterminer leur exploitabilité réelle dans le contexte spécifique du client.
- Prioriser les vulnérabilités selon un modèle de risque pertinent.
- Préparer la phase d’exploitation, en sélectionnant les vecteurs les plus prometteurs.

Cette étape transforme les données brutes obtenues lors de la reconnaissance en informations exploitables pour les phases offensives.

2. Approche méthodologique

L'analyse des vulnérabilités doit être structurée en plusieurs phases :

2.1 Phase 1 : Consolidation des données issues de la reconnaissance

À partir des informations collectées précédemment :

- Adresses IP identifiées,
- Services et ports ouverts,
- Versions d'application,
- Empreintes technologiques,
- Composants cloud ou SaaS,
- Utilisateurs ou comptes exposés.

On crée un inventaire exploitable des actifs scannés :
↳ serveurs, applications web, réseaux internes, systèmes industriels, etc.

Cette cartographie est essentielle : une analyse de vulnérabilité mal cadrée peut entraîner des faux positifs, des oublis, ou des analyses réalisées sur les mauvais systèmes.

2.2 Phase 2 : Scan automatisé de vulnérabilités

Les scanners permettent de :

- Détecter des CVE connues,
- Identifier des services mal configurés,

- Repérer des versions vulnérables,
- Analyser des failles de configurations réseau.

Outils typiques (sans rentrer dans les détails techniques)

- Scanners réseau,
- Scanners de vulnérabilités web,
- Modules de reconnaissance de Framework.

Limites des scanners

- Nombreux faux positifs,
- Peu de contextualisation,
- Incapacité à détecter des vulnérabilités logiques,
- Dépendance à des bases de signatures.

Les scans doivent donc être considérés comme un point de départ, jamais comme un résultat final.

2.3 Phase 3 : Vérification manuelle et réduction des faux positifs

L'intervention humaine est indispensable pour :

- Valider ou invalider les vulnérabilités détectées,
- Analyser les prérequis de l'exploitation,
- Déterminer si la vulnérabilité est exploitable dans le contexte réel,
- Identifier des failles logiques non détectables automatiquement (authentification cassée, contournements, erreurs métier).

Exemples de validations manuelles :

- Test des protections WAF,
- Tests d'accès non autorisés,

- Analyse des permissions internes,
- Vérification des dépendances vulnérables exploitées.

2.4 Phase 4 : Analyse contextuelle

Cette étape permet de déterminer si une vulnérabilité est *vraiment* exploitable.

Elle prend en compte :

- Les droits disponibles,
- Les configurations réseau,
- La segmentation,
- La présence ou non de défenses (WAF, IDS, MFA…),
- L'importance de l'actif dans l'organisation.

Exemple :
Une CVE critique sur un serveur interne isolé peut représenter un risque moindre qu'une CVE moyenne sur un serveur exposé publiquement contenant des données sensibles.

2.5 Phase 5 : Priorisation des vulnérabilités

La priorisation doit se baser sur plusieurs critères :

- Impact potentiel (confidentialité, intégrité, disponibilité)
- Probabilité d'exploitation
- Existence d'exploits publics
- Niveau d'accès nécessaire
- Position du système dans l'architecture

- Valeur métier de l’actif

Des modèles de notation peuvent être utilisés :

- CVSS (Common Vulnerability Scoring System),
- OWASP Risk Rating Methodology,
- MITRE ATT&CK (pour contextualisation offensive),
- Modèles internes orientés métier.

2.6 Phase 6 : Sélection des vulnérabilités pour l’exploitation

Toutes les vulnérabilités identifiées ne sont pas nécessairement exploitées pendant un test d’intrusion.

Le testeur sélectionne celles qui :

- Sont réellement exploitables,
- Apportent un gain opérationnel (élévation de privilèges, pivot, accès),
- Correspondent aux objectifs du test,
- N’engendrent pas un risque trop élevé pour le système (en accord avec les règles d’engagement).

Certaines vulnérabilités critiques mais instables peuvent être exclues si elles peuvent provoquer une dégradation de service.

3. Types de vulnérabilités analysées

L’analyse peut porter sur de nombreuses catégories, par exemple :

3.1 Vulnérabilités système

- OS obsolètes
- Services ouverts inutilement

- Problèmes d’authentification

3.2 Vulnérabilités réseau

- SNMP exposé
- SSL/TLS faibles
- NAT et firewalls mal configurés

3.3 Vulnérabilités applicatives

- Injection (SQL, XSS…)
- Contrôles d’accès insuffisants
- Exposition de données sensibles

3.4 Vulnérabilités cloud

- Permissions IAM trop permissives
- Buckets S3 ouverts
- Clés d’API exposées

3.5 Vulnérabilités humaines

- Comptes compromis,
- Mots de passe faibles,
- Informations sensibles publiées en ligne.

4. Importance de la documentation

L’analyse des vulnérabilités doit être parfaitement documentée, incluant :

- Problème identifié,
- Description détaillée,
- Contexte d’exploitation,

- Preuves (captures, requêtes...),
- Impact potentiel,
- Méthode de validation,
- Suggestion de remédiation.

Cette documentation est essentielle pour permettre au client :

- De comprendre l'impact réel,
- De reproduire éventuellement les tests,
- D'appliquer des correctifs adaptés et efficaces.

5. Erreurs courantes à éviter

- Se reposer uniquement sur des scanners automatisés.
- Considérer toutes les CVE comme exploitables sans validation.
- Omettre les vulnérabilités logiques.
- Sous-estimer la valeur métier des systèmes.
- Continuer à scanner en dehors du périmètre autorisé.
- Oublier de documenter les fausses pistes et les exclusions.

Exploitation :

1. Objectifs de la phase d’exploitation

L’exploitation vise à :

- Valider qu'une vulnérabilité est effectivement exploitable.
- Démontrer l'impact concret d'une faille (accès, fuite de données, élévation de privilèges…).
- Obtenir un point d'appui initial sur le système.
- Évaluer la robustesse des mesures de défense.
- Progresser vers les objectifs définis avec le client (compromission d'un domaine, d'un compte privilégié, d'un système critique…).

L'exploitation doit être **contrôlée, proportionnée et non perturbatrice**.

2. Approche méthodologique

La méthodologie d'exploitation est organisée en plusieurs étapes pour garantir précision, sécurité et traçabilité.

2.1 Préparation de l'exploitation

Avant toute tentative, l'auditeur :

- Sélectionne les vulnérabilités jugées exploitables,
- Analyse les prérequis (accès réseau, permissions, contexte applicatif),
- Choisit les vecteurs d'attaque les plus appropriés,
- Prépare ou adapte les scripts ou charges utiles nécessaires,
- Valide que l'exploitation ne présente pas un risque de dégradation (DoS, corruption de données, surcharge système).

Cette phase est cruciale pour éviter des impacts involontaires.

2.2 Exploitation technique des vulnérabilités

L’auditeur tente ensuite d’exploiter les vulnérabilités en s’appuyant sur :

- **Exploitation des vulnérabilités réseau**
  - Exploitation de services obsolètes,
  - Contournement d’authentification,
  - Failles dans les protocoles,
  - Mauvaise configuration des firewalls ou routeurs.
- **Exploitation applicative**
  - Injections (SQL, LDAP, OS…),
  - Faille de gestion d’accès ou de logique métier,
  - Exécution de code à distance,
  - Contournement ou altération de sessions.
- **Exploitation des vulnérabilités système**
  - Services vulnérables à l’escalade de privilèges,
  - Failles de configuration,
  - Utilisation de chemins mal sécurisés.
- **Exploitation du Cloud**
  - Accès non autorisé via clés API exposées,
  - Abus de permissions IAM,
  - Escalade latérale entre ressources.
- **Exploitation humaine / ingénierie sociale**

(Uniquement si autorisée dans les règles d’engagement)

  - Hameçonnage contrôlé,
  - Récupération d’identifiants exposés,

- Tests de sensibilisation.

2.3 Adaptation et exploitation manuelle

L'exploitation doit être modulée en fonction des mécanismes défensifs rencontrés :

- Filtrages réseau,
- Web Application Firewall (WAF),
- IDS/IPS,
- MFA,
- Segmentation,
- Journaux renforcés.

Les attaques sont adaptées pour :

- Contourner les protections,
- Réduire la signature (low & slow),
- Utiliser des techniques alternatives (logiques plutôt que techniques, exploitation via chaînes de vulnérabilités, etc.).

Cette partie demande une forte expertise humaine.

2.4 Validation de la compromission

Une exploitation n'est considérée comme réussie que si l'auditeur obtient :

- Une exécution de commande,
- Un accès non autorisé (à une interface, un service, une API),
- Une fuite de données exploitables,
- Une élévation de privilèges,

- Un accès persistant ou un pivot.

Chaque action doit être **documentée précisément**, incluant :

- La technique utilisée,
- Le résultat obtenu,
- La preuve (captures d'écran, logs, sortie de commande…),
- Les conditions d'exploitation.

3. Progression post-exploitation (si autorisée)

Une fois un accès gagné, l'étape suivante consiste à mesurer l'impact réel via des actions contrôlées.

3.1 Énumération post-compromission

- Inventaire des privilèges,
- Services disponibles localement,
- Mots de passe ou clés stockées,
- Architecture du réseau interne.

3.2 Escalade de privilèges

- Exploitation locale,
- Abus de permissions,
- Mauvaises configurations.

3.3 Mouvement latéral

- Accès à d'autres machines,
- Rebonds via interfaces internes ou API,
- Exploitation de relations de confiance.

3.4 Établissement de la persistance

Uniquement si le cadre l'autorise, par exemple :

- Création d'un compte temporaire,
- Ajout d'une clé SSH,
- Token cloud temporaire.

Cette étape permet de reproduire un scénario d'attaquant réaliste mais reste fortement encadrée pour ne pas altérer le système.

4. Cadre éthique et sécurité

L'exploitation doit se dérouler :

• Selon les règles d'engagement

- Périmètre autorisé,
- Plages horaires,
- Restrictions (pas de DoS, pas de modification de données sensibles, etc.).

• En minimisant les risques

- Pas de perturbation,
- Pas d'altération des données,
- Tests en environnement miroir si nécessaire.

• Avec traçabilité complète

Chaque tentative est enregistrée :

- Actions,
- Outils,
- Résultats,

- Heures,
- Systèmes concernés.

5. Documentation et preuves

Un rapport d'exploitation doit inclure :

- La vulnérabilité exploitée,
- La méthodologie,
- Les prérequis nécessaires,
- Les étapes détaillées,
- Les preuves,
- L'impact démontré,
- Les recommandations de remédiation,
- Une estimation du niveau de risque.

L'exploitation doit être reproductible par le client s'il souhaite valider l'impact.

6. Erreurs à éviter

- Tenter d'exploiter des vulnérabilités instables sans validation préalable,
- Exécuter des charges inutiles ou dangereuses,
- Dépasser le périmètre autorisé,
- Ignorer les mécanismes défensifs,
- Oublier de documenter clairement les étapes.

Post-Exploitation :

1. Objectifs de la post-exploitation

La phase de post-exploitation permet :

- D'évaluer la valeur et la sensibilité des données accessibles après compromission ;
- De mesurer l'ampleur d'un impact sur les opérations ;
- De comprendre comment un attaquant pourrait se déplacer dans l'environnement ;
- D'évaluer la réaction des équipes de sécurité et les capacités de détection ;
- De valider les scénarios de menace identifiés en pré-engagement.

Cette phase **n'est pas systématiquement réalisée** : elle dépend du périmètre autorisé.

2. Approche méthodologique

La méthodologie de post-exploitation se concentre sur quatre axes :

1. **Collecte d'informations internes**
2. **Élévation de privilèges**
3. **Mouvements latéraux**
4. **Maintien d'accès (si autorisé)**

3. Collecte d’informations internes

Une fois un accès initial obtenu (poste utilisateur, serveur, application…), le testeur analyse l’environnement local et réseau pour :

3.1 Inventaire local

- Privilèges du compte compromis,
- Services et processus en cours,
- Configuration système,
- Logs accessibles,
- Informations d’identification stockées (cookies, tokens, clés API, mots de passe sauvegardés…),
- Points de montage réseau,
- Applications installées et versions.

3.2 Cartographie du réseau interne

- Découverte d’autres machines,
- Analyse du routage interne,
- Détection de domaines, forêts AD ou sous-réseaux Cloud,
- Identification de serveurs critiques.

3.3 Découverte des données sensibles

Toujours en mode **lecture uniquement**, conformément aux règles éthiques.

Exemples :

- Données personnelles ou financières,
- Propriété intellectuelle,
- Journaux internes exposant des secrets techniques,

- Partages de fichiers mal sécurisés.

Cette étape permet de mesurer **l'impact potentiel d'une fuite de données**.

4. Élévation de privilèges

L'objectif est de déterminer si un attaquant pourrait obtenir :

- Des privilèges système/administrateur,
- Des droits cloud élevés (IAM Admin, Owner…),
- Des privilèges AD (Domain Admin),
- L'accès à des services critiques.

Méthodes évaluées (sans détails dangereux)

- Mauvaises configurations système,
- Permissions excessives,
- Héritages mal maîtrisés (ACL, IAM),
- Gestion des mots de passe défaillante,
- Chaînes de vulnérabilités.

Le but est d'établir **ce qui est possible**, pas d'enseigner comment le faire techniquement.

5. Mouvement latéral

Le mouvement latéral consiste à analyser la capacité d'un attaquant à :

- Rebondir d'une machine à une autre,
- Utiliser des relations de confiance internes,
- Exploiter des sessions ouvertes ou des clés stockées,

- Atteindre des systèmes critiques.

Objectifs :

- Mesurer la propagation possible d'une compromission,
- Simuler le comportement d'un attaquant réel (APTs, ransomware…),
- Vérifier la segmentation réseau,
- Valider les contrôles Zéro Trust (ou leur absence).

Lors de cette étape, **toute action intrusive doit être strictement contrôlée** pour éviter toute perturbation du SI.

6. Maintien d'accès (Persistance)

(Uniquement si autorisé et toujours sans installer de backdoors réelles)

La persistance vise à évaluer **si un attaquant pourrait rester dans le système après détection ou redémarrage**.

Les techniques évaluées peuvent inclure :

- Exploitation d'un compte à privilèges déjà existant (jamais création incontrôlée de comptes),
- Utilisation de mécanismes applicatifs légitimes (tokens longue durée, clés API),
- Abus de configurations persistantes déjà présentes.

Dans le cadre d'un test d'intrusion, l'auditeur **ne déploie jamais** :

- Implants,
- Malwares,
- Backdoors,
- Mécanismes clandestins non autorisés.

La persistance doit rester **propre, transparente et réversible**.

7. Analyse de l’impact et scénarios réalistes

Une fois les actions post-exploitation réalisées, l’auditeur évalue explicitement :

- Ce que l’attaquant aurait pu faire avec les accès obtenus,
- Quelles données auraient pu être exfiltrées,
- Quelles ressources auraient pu être sabotées,
- Quel serait l’impact métier (financier, réglementaire, réputationnel).

Les scénarios sont alors traduits en **risques concrets**, compréhensibles pour les dirigeants.

8. Éléments de détection et réponse

La post-exploitation permet aussi de mesurer la capacité de défense :

8.1 Détection

- Les journaux ont-ils enregistré les actions ?
- Les alertes ont-elles été remontées ?
- Les mécanismes SIEM/EDR ont-ils réagi ?

8.2 Réponse

- Les équipes Blue Team ont-elles remarqué l’activité ?
- Une action de confinement a-t-elle été engagée ?
- Combien de temps avant la détection ?

Cela permet un **test des capacités SOC / CSIRT**, souvent inattendu pour le client.

9. Documentation des actions

Chaque étape doit être documentée pour permettre au client :

- De comprendre comment l'impact réel a été démontré,
- De reproduire l'analyse,
- De corriger les failles,
- D'améliorer ses défenses.

La documentation inclut toujours :

- Étapes réalisées,
- Accès obtenus,
- Preuves contrôlées,
- Risques associés,
- Mesures correctives.

Rapport :

1. Objectifs du reporting

Le reporting vise à :

- **Documenter l'ensemble des résultats** du test d'intrusion.
- **Démontrer l'impact réel** des vulnérabilités et compromissions.
- **Permettre au client d'agir concrètement** via des recommandations détaillées.
- **Communiquer un état clair du niveau de risque** du système testé.
- **Fournir les éléments de conformité** nécessaires (ISO 27001, PCI-DSS, réglementations locales…).
- **Assurer une traçabilité complète** des actions réalisées.

Le rapport doit satisfaire **à la fois les parties prenantes techniques et non techniques**.

2. Structure recommandée d'un rapport de test d'intrusion

Un bon rapport s'organise généralement en deux grands ensembles :

1. **Le rapport exécutif (Executive Summary)**
2. **Le rapport technique détaillé**

Chaque partie répond à des besoins différents et doit être traitée comme un livrable autonome.

3. Rapport exécutif (Executive Summary)

Le rapport exécutif est un document **stratégique**, non technique, destiné à la direction, au RSSI, au GRC ou aux décideurs métier.

Il doit être :

- Court,
- Clair,
- Visuel,
- Axé sur les risques et impacts réels,
- Dénué de jargon technique.

Il contient généralement :

3.1 Objectifs de la mission

(En termes métier, pas techniques)

3.2 Méthodologie utilisée

– normes ou cadres (OSSTMM, PTES, NIST, OWASP…)
– périmètre testé
– limites éventuelles

3.3 Résumé global des résultats

– résumé des vulnérabilités majeures
– description synthétique des compromissions réussies
– conséquences potentielles pour l'entreprise

3.4 Cartographie des risques

– classement par criticité
– score de risque (CVSS ou interne)
– vue sous forme de graphique / heatmap

3.5 Niveau global de sécurité

– évaluation qualitative (faible / moyen / élevé)
– comparaison éventuelle avec les standards attendus

3.6 Recommandations stratégiques prioritaires

– priorisation
– estimation d'effort / de complexité
– pistes d'amélioration à court, moyen et long terme

Ce résumé doit permettre à un cadre dirigeant de comprendre **l'urgence**, **l'impact** et **la priorisation**.

4. Rapport technique détaillé

C'est ici que l'ensemble du travail est exposé de manière complète, reproductible et méthodique.

Il est destiné aux équipes :

- Sécurité,
- Réseau,
- Infrastructure,
- Applications,
- Cloud,
- SOC/CSIRT.

Le rapport détaillé inclut :

4.1 Description du périmètre

- Systèmes, applications, réseaux testés
- Exclusions
- Contraintes (horaires, interdictions d'exploitation dangereuse…)

- Dates de l’audit

4.2 Méthodologie et outils

Transparence totale :

- Phases du test décrites (collecte, scan, analyse, exploitation, post-exploitation, etc.)
- Outils utilisés (sans fournir de code ou techniques illégales)
- Limitations connues

4.3 Résultats globaux

Un tableau récapitulatif listant :

- Nom de la vulnérabilité
- Classification (CVE si applicable)
- Criticité
- Impact
- Système concerné
- Difficulté d’exploitation
- Recommandation rapide

4.4 Détails des vulnérabilités

Pour chaque vulnérabilité, on fournit :

a. Titre clair et précis

Ex : « Absence de contrôle d’accès sur l’API /admin »

b. Description

Contexte de la vulnérabilité.

c. Impact

(Technique et métier)

d. Scénario démontré

Ce que l'auditeur a démontré **dans le cadre autorisé** :

- Accès non autorisé,
- Élévation de privilèges,
- Exfiltration simulée,
- Mouvement latéral, etc.

e. Preuves

Captures, logs, observations (toujours non destructives).

f. Analyse du risque

Selon un modèle choisi :

- CVSS,
- DREAD,
- Risque interne organisationnel.

g. Recommandations détaillées

Toujours actionnables :

- Patch à appliquer,
- Configuration spécifique,
- Renforcement des accès,
- Bonne pratique,

- Mécanisme de détection.

h. Indicateurs de détection

Très appréciés des équipes SOC :

- Quels logs surveiller,
- Quels signaux sont révélateurs,
- Comment détecter ce type d'activité.

5. Annexes techniques

Les annexes permettent de documenter sans surcharger le rapport :

- Liste complète des IP scannées,
- Liste des outils et versions,
- Scripts utilisés (si autorisés par contrat),
- Références normatives,
- Logs non critiques,
- Schémas réseau reconstitués.

6. Qualité du reporting : bonnes pratiques

Un bon rapport de test d'intrusion doit être :

✓ Clair

Compréhensible même par un non-technicien.

✓ Exact

Pas d'exagération de risque, pas de minimisation non plus.

✓ **Reproductible**

Les équipes doivent pouvoir rejouer les tests en interne.

✓ **Actionnable**

Les recommandations doivent être concrètes et applicables.

✓ **Structurellement cohérent**

Même logique pour chaque vulnérabilité.

✓ **Sécurisé**

Aucune donnée sensible en clair (mots de passe, clés privées…).

✓ **Professionnel**

Ton neutre et factuel.

7. Sécurité et confidentialité du rapport

Le rapport contient des informations très sensibles.
Il doit être :

- Chiffré (PGP, ZIP AES, portail sécurisé)
- Transmis uniquement aux destinataires autorisés
- Versionné
- Conservé selon les règles contractuelles
- Supprimé en fin de mission si le contrat l'exige

Le rapport final est souvent considéré comme un document **hautement confidentiel**.

8. Présentation de fin de mission

En plus du rapport, une présentation orale (debriefing) est presque toujours réalisée :

- Revue des vulnérabilités critiques,
- Explication des scénarios,
- Validation des recommandations,
- Priorisation des actions,
- Réponses aux questions techniques.

Ce moment permet d’assurer une bonne appropriation des résultats par le client.